

LAB 2 REPORT: WISCKEY VS LEVELDB

1. Design Summary

In this lab, we implemented a **WiscKey-like** design on top of LevelDB to improve write performance for large values.

- **Baseline (LevelDB):** Key and value are stored **together** inside LevelDB SSTables.
- **WiscKey Design:**
 - **Keys** are stored in LevelDB with an **offset** (pointer) into a **separate value log** (valuelog.dat).
 - **Values** are stored separately in a log file, avoiding large values inflating LevelDB structures.
- Only the wisckey_test.cc file was modified. LevelDB base code and lab2_common.h were reused without modification.

This design reduces write amplification when values are large.

2. Performance Results

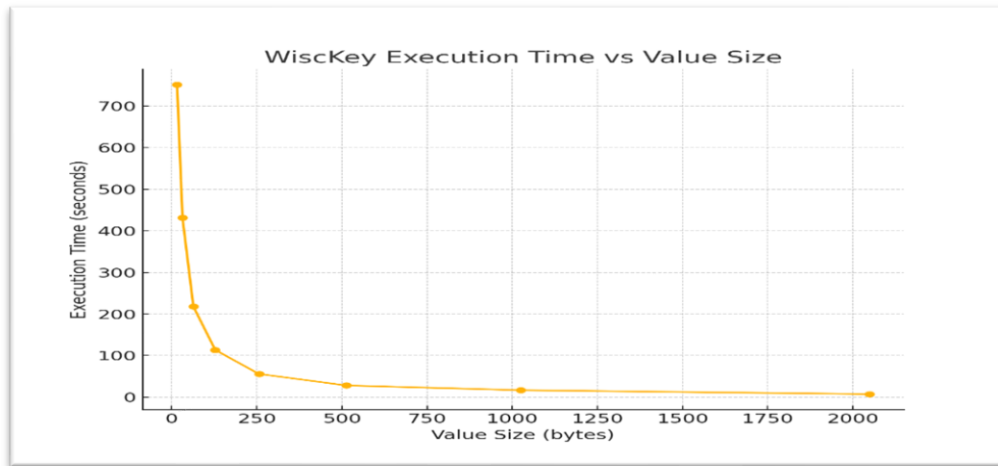
Value Size (bytes)	LevelDB Time (s)	WiscKey Time (s)
16	630.165	751.606
32	410.862	431.404
64	263.676	217.868
128	195.018	113.844
256	150.144	56.1367
512	129.891	28.3861
1024	115.721	17.0096
2048	106.766	7.39692

LAB 2 REPORT: WISKEY VS LEVELDB

3. Results and Charts

(a) WiscKey Execution Time vs Value Size

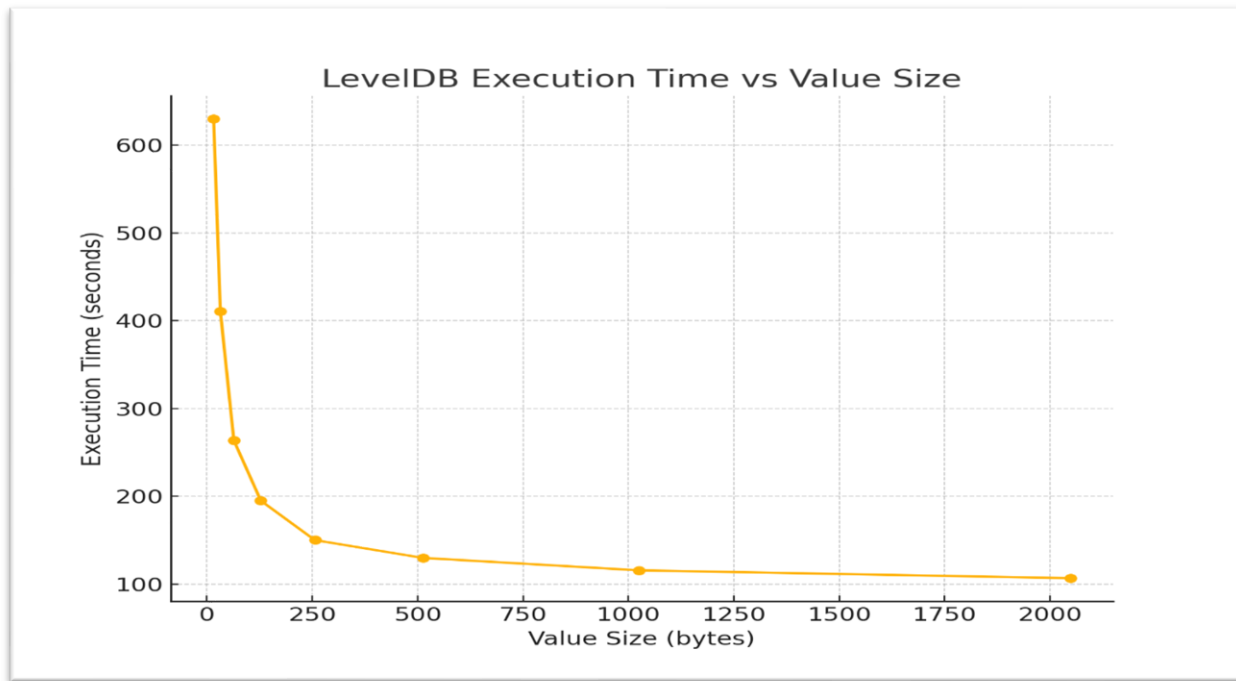
- WiscKey performance improves dramatically with larger value sizes.
- Sequential writes to value log avoid frequent LSM-tree updates.



(b) LevelDB Execution Time vs Value Size

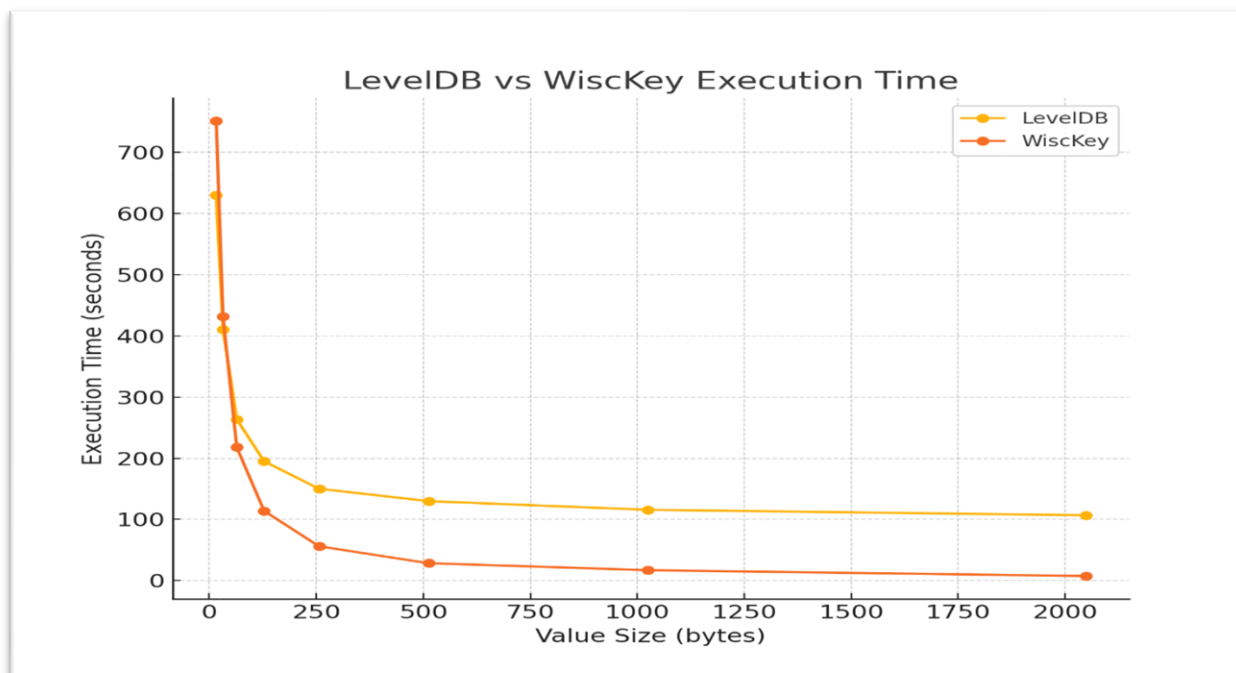
- LevelDB handles **small values faster** due to direct storage inside the LSM-tree.
- However, as values get bigger, LevelDB's execution time worsens due to **higher write amplification**.

LAB 2 REPORT: WISCKEY VS LEVELDB



(c) Comparison: WiscKey vs LevelDB

- **For small value sizes (16, 32, 64 bytes):** LevelDB is faster because overheads (pointer management, extra file access) dominate WiscKey performance.
- **For larger value sizes (≥ 128 bytes):** WiscKey is significantly faster as it decouples value movement from the tree structure, avoiding large compaction overheads.



LAB 2 REPORT: WISCKEY VS LEVELDB

4. Analysis and Observations

- **Small Value Sizes:**

LevelDB outperforms WiscKey because:

- LSM-tree is compact.
- No extra file I/O for fetching values.
- Pointer overhead in WiscKey adds latency for tiny payloads.

- **Large Value Sizes:**

WiscKey significantly outperforms because:

- Appending large values sequentially is much cheaper.
- LevelDB suffers from expensive compactions when values are embedded inside the tree.

5. Conclusion:

WiscKey design becomes more advantageous as value sizes increase. It offers a scalable solution for write-heavy and large-value key-value workloads.